



Universidad Politécnica de Madrid

Bases de Datos II

Práctica Neo4j: Jotun's Lair

Consultas Cypher, Graph Data Science y recomendaciones de diseño

Yixuan Lu Guo
Shengkai Zhu

21 de mayo de 2026

Índice

1. Introducción	2
2. Entorno de ejecución	2
3. Modelo de datos	2
4. Consultas Cypher	3
4.1. Consulta 1: objetos sin forma directa de obtención	3
4.2. Consulta 2: recetas con mayor ganancia	4
4.3. Consulta 3: arma crafteable con mayor daño de fuego	4
4.4. Consulta 4: arma con mayor daño combinado por tipo	5
4.5. Consulta 5: proceso para obtener el arma 297	5
4.6. Consulta 6: monstruos para fabricar el arma 880	6
4.7. Consulta 7: proceso de fabricación más complejo	6
4.8. Consulta 8: arma que requiere matar más monstruos distintos	7
4.9. Consulta 9: debilidad máxima por localización	8
4.10. Consulta 10: recomendación de arma por localización	8
5. Análisis de grafos con Graph Data Science	10
5.1. Paso 1: creación de relaciones SIMILAR con índice de Jaccard	10
5.2. Paso 2: proyección del grafo y detección de comunidades con Leiden	10
5.3. Paso 3: PageRank por comunidad	11
6. Recomendaciones de diseño	13
6.1. Cardinalidad	13
6.2. Patrones de consulta	13
6.3. Identidad y conectividad	14
6.4. Evolución del modelo	14
6.5. Conclusión: cuando está justificado el cambio	14
7. Conclusiones	14

1. Introducción

Esta práctica se centra en el uso de **Neo4j** para modelar y consultar un sistema de crafeo de armás del videojuego ficticio *Jotun's Lair*. En este dominio, el jugador obtiene materiales cazando monstruos, crea armas directamente mediante procesos de crafeo y también puede mejorar armas mediante procesos de actualización.

El uso de una base de datos de grafos resulta adecuado porque el problema contiene muchas relaciones naturales entre entidades: armas, monstruos, materiales, recetas, localizaciones y elementos. Además, las armas no solo tienen daño base, sino también posibles daños elementales, mientras que los monstruos pueden ser débiles o resistentes a diferentes elementos.

La práctica se divide en tres partes principales:

- Realización de consultas Cypher sobre el grafo.
- Análisis de grafos con *Graph Data Science* para agrupar armas similares.
- Reflexión de diseño sobre si conviene modelar los tipos de armas como nodos.

2. Entorno de ejecución

La base de datos se ha ejecutado mediante Docker, usando Neo4j 5.25 y los plugins **APOC** y **Graph Data Science**. Para cargar el dump inicial de la base de datos se utiliza:

```
docker run -it --rm \
  --volume="$PWD/data:/data" \
  --volume="$PWD/import:/backups" \
  neo4j:5.25 \
  neo4j-admin database load neo4j \
  --from-path=/backups --overwrite-destination=true --verbose
```

Después, Neo4j se arranca con:

```
docker run --rm \
  --name neo4j-jotuns \
  --volume="$PWD/data:/data" \
  --volume="$PWD/plugins:/plugins" \
  --publish=7474:7474 \
  --publish=7687:7687 \
  --env NEO4J_AUTH=neo4j/password \
  --env NEO4J_PLUGINS='["apoc","graph-data-science"]' \
  neo4j:5.25
```

La interfaz web queda disponible en <http://localhost:7474> con usuario neo4j y contraseña password.

3. Modelo de datos

El grafo está formado principalmente por los siguientes tipos de nodos:

- **Weapon**: armás del juego.
- **WeaponCraft**: proceso de creación directa de un arma.
- **WeaponUpgrade**: proceso de mejora o actualización de un arma.
- **Item**: materiales u objetos.

- **Recipe**: recetas para producir items.
- **Monster**: monstruos que pueden dropear materiales.
- **Location**: zonas o biomas donde viven los monstruos.
- **Element**: elementos de daño o estado.

Las relaciones principales del modelo son:

- **PRODUCES**: indica que una receta o proceso produce un item o arma.
- **NEEDS**: indica que materiales necesita una receta, crafteo o mejora.
- **TRANSFORMS**: indica que una mejora transforma un arma en otra.
- **DROPS**: indica que materiales puede soltar un monstruo.
- **LIVES_IN**: indica en que localización vive un monstruo.
- **WEAK_TO**: indica la debilidad de un monstruo frente a un elemento.
- **RESISTENT_TO**: indica la resistencia de un monstruo frente a un elemento.
- **DEALS**: indica el daño elemental extra que causa un arma.

4. Consultas Cypher

En esta sección se explican las diez consultas Cypher desarrolladas sobre la base de datos. Cada consulta responde a una pregunta concreta del enunciado.

4.1. Consulta 1: objetos sin forma directa de obtención

```
MATCH (i:Item)
WHERE NOT EXISTS { (:Recipe)-[:PRODUCES]->(i) }
AND NOT EXISTS { (:Monster)-[:DROPS]->(i) }
AND (
  EXISTS { (:WeaponCraft)-[:NEEDS]->(i) }
  OR EXISTS { (:WeaponUpgrade)-[:NEEDS]->(i) }
)
RETURN i.name AS nombre, i.description AS descripcion
```

Esta consulta busca objetos que se utilizan para crear o mejorar armas pero que no se pueden obtener ni mediante recetas ni matando monstruos. Para ello se parte de los nodos **Item** y se descartan los que aparecen como salida de una receta o como drop de un monstruo. Después se comprueba que el item si sea necesario en algun **WeaponCraft** o **WeaponUpgrade**. Es importante envolver la última condición con parentesis para que Neo4j no interprete mal la prioridad lógica entre AND y OR.

Resultado: la consulta devuelve **29 items**. Aparecen materiales como **Carbalita**, **Cristal de tierra**, **Hueso misterioso**, **Piedra de fuego** o **Fucium**. Todos son minerales, huesos o materiales base que el modelo no representa como fabricables ni como drops, pero que sí son necesarios en procesos de crafteo o mejora. Esto refleja que en el juego existen materiales que simplemente se recolectan en el mundo (minas, nidos) sin pasar por combate ni fabricación, y que el grafo los trata como inputs externos al sistema modelado.

4.2. Consulta 2: recetas con mayor ganancia

```
MATCH (r:Recipe)-[p:PRODUCES]->(output:Item)
MATCH (r)-[:NEEDS]->(input:Item)

WITH r,
     collect(input) AS inputs,
     output,
     p.amount AS outAmount

WHERE ALL(i IN inputs WHERE NOT EXISTS { (:Recipe)-[:PRODUCES]->(i) })

WITH r,
     output,
     outAmount,
     inputs,
     reduce(
         coste = 0,
         i IN inputs | coste + coalesce(i.value, 0)
     ) AS costeTotal

WITH r,
     [i IN inputs | i.name] AS itemsInput,
     output.name AS itemOutput,
     coalesce(output.value, 0) * coalesce(outAmount, 1) - costeTotal AS
     ganancia

RETURN r.id AS recetaId,
       itemsInput,
       itemOutput,
       ganancia
ORDER BY ganancia DESC
LIMIT 5
```

Esta consulta calcula las cinco recetas más rentables. La parte clave es usar `collect(input)` antes del filtro con `ALL`: si se filtrase input por input, una receta podría colarse aunque solo uno de sus materiales cumpliera la condición. Con `ALL` se obliga a que todos los inputs de la receta no sean producidos por otras recetas. El coste total se calcula con `reduce` y la ganancia es `valor del output * cantidad - coste total de inputs`.

Resultado: la receta más rentable es la **244**, que produce **Fuente de vida** con una ganancia de **530**, bastante por encima del resto. Le siguen **Polvo de cascara**, **Polvo de vida**, **Trampa eléctrica** y **Polvo de demonio**. Todos son consumibles o objetos útiles que se generan a partir de pocos materiales baratos. La diferencia tan grande entre la receta 244 y las demás sugiere que es una receta especialmente eficiente o que **Fuente de vida** tiene un valor de mercado muy alto en relación a sus componentes.

4.3. Consulta 3: arma crafteable con mayor daño de fuego

```
MATCH (:Element {name:'fire'})<-[d:DEALS]-(w:Weapon)<-[p:PRODUCES]-(wc:
    WeaponCraft)

WITH w, wc, d
ORDER BY d.damage DESC
LIMIT 1

MATCH (wc)-[:NEEDS]->(i:Item)
```

```

OPTIONAL MATCH (m:Monster)-[:DROPS]->(i)

WITH w,
    collect(DISTINCT m.name) AS monstersRaw,
    collect(DISTINCT CASE WHEN m IS NULL THEN i.name END) AS extrasRaw

RETURN w.name AS nombre,
    w.kind AS tipo,
    [m IN monstersRaw WHERE m IS NOT NULL] AS monstruos,
    [e IN extrasRaw WHERE e IS NOT NULL] AS materialesExtra

```

Primero se localiza el arma con mayor daño de fuego que sea crafteable directamente, partiendo desde `Element {name: 'fire'}` y siguiendo `DEALS` hasta `Weapon`, asegurandonos de que viene de un `WeaponCraft`. Una vez elegida, se revisan sus materiales y se separan entre los que provienen de monstruos y los que no (materiales extra). Se usa `OPTIONAL MATCH` para no perder materiales sin monstruo asociado, y `CASE WHEN m IS NULL` para capturarlos como extras.

Resultado: el arma es **Decapitagallinas I**, de tipo **great-sword**. Para fabricarla hay que matar a **Yian Kut-Ku**, y además hace falta el material extra **Carbalita**, que no sale de ningún monstruo (ya aparecía en la consulta 1 como item sin forma directa de obtención). Resulta interesante que el arma más fuerte de fuego crafteable dependa solo de un monstruo y de un mineral externo.

4.4. Consulta 4: arma con mayor daño combinado por tipo

```

MATCH (w:Weapon)
OPTIONAL MATCH (w)-[:DEALS]->(:Element {kind: 'element'})

WITH w,
    coalesce(w.damage, 0) + coalesce(sum(d.damage), 0) AS danoTotal

WITH w.kind AS tipo,
    max(danoTotal) AS maxDano,
    collect({nombre: w.name, dano: danoTotal}) AS armas

RETURN tipo,
    [a IN armas WHERE a.dano = maxDano | a.nombre] AS armas,
    maxDano AS danoCombinado

ORDER BY danoCombinado DESC

```

Se calcula el daño total de cada arma como la suma del daño base más los daños elementales de tipo `element`. Se usa `OPTIONAL MATCH` para incluir armas sin daño elemental y `coalesce` para que esos casos cuenten como 0. Después se agrupa por tipo y se devuelven todas las armas que alcanzan el máximo, gestionando posibles empates con comprensión de listas.

Resultado: la consulta devuelve los **14 tipos de arma** del juego. Los mayores daños combinados corresponden a **great-sword** y **gunlance**, ambos con **290 de daño combinado**. La consulta gestiona correctamente los empates: por ejemplo, en **lance** aparecen dos armas con daño combinado de **260**. Esto confirma que el enfoque de recopilar todas las armas con `collect` y filtrar después con comprensión de listas es el correcto para este tipo de consultas en Cypher.

4.5. Consulta 5: proceso para obtener el arma 297

```

MATCH path = (w2:Weapon {id:297})<-[:TRANSFORMS*]-(w1:Weapon)
WHERE EXISTS { (w1)<-[:PRODUCES]-(:WeaponCraft) }

```

```
WITH [n IN nodes(path) WHERE n:Weapon | n.id] AS weapons
RETURN weapons AS proceso
```

La consulta reconstruye el proceso de actualización del arma con id 297. Se busca un camino de relaciones TRANSFORMS* desde el arma final hacia atrás, hasta un arma que sea crafteable directamente (EXISTS sobre WeaponCraft). Como el patrón está escrito empezando desde el arma final hacia la base, nodes(path) ya devuelve los nodos en el orden correcto sin necesidad de reverse.

Resultado: el proceso obtenido es exactamente [297, 296, 295], que coincide con el ejemplo del enunciado. Esto significa que hay que empezar fabricando el arma 295 con un WeaponCraft, luego actualizarla a 296 y finalmente a 297. El hecho de que solo haya dos pasos de actualización lo convierte en un proceso relativamente corto comparado con el de la consulta 7.

4.6. Consulta 6: monstruos para fabricar el arma 880

```
MATCH path = (base:Weapon) -[:TRANSFORMS*]->(target:Weapon {id: 880})
WHERE EXISTS { (:WeaponCraft) -[:PRODUCES]->(base) }

WITH path
ORDER BY length(path) ASC
LIMIT 1

WITH [n IN nodes(path) WHERE n:Weapon | n] AS armas

UNWIND armas AS arma

OPTIONAL MATCH (arma) <-[:PRODUCES]-(wc:WeaponCraft) -[:NEEDS]->(:Item)
<-[:DROPS]-(m1:Monster)
OPTIONAL MATCH (arma) <-[:TRANSFORMS]-(wu:WeaponUpgrade) -[:NEEDS]->(:Item)
<-[:DROPS]-(m2:Monster)

WITH collect(m1.name) + collect(m2.name) AS monstersRaw

UNWIND monstersRaw AS monster

WITH DISTINCT monster
WHERE monster IS NOT NULL

RETURN monster AS monstruo
ORDER BY monstruo
```

Esta consulta busca todos los monstruos que hay que matar para completar la cadena de fabricación del arma 880. Lo importante es no mirar los nodos Weapon directamente para buscar materiales, sino los nodos que tienen relaciones NEEDS: WeaponCraft y WeaponUpgrade. Se toma el camino más corto hacia el arma objetivo y se procesan todos los pasos de creación y mejora por separado con UNWIND y OPTIONAL MATCH.

Resultado: la consulta devuelve **6 monstruos distintos**: Gravios, Gypceros, Jin Dhaad, Lala Barina, Nerscylla y Yian Kut-Ku. Considerando todo el proceso de creación y mejora hasta el arma 880, estos son los únicos monstruos que aportan materiales necesarios. Se probó también una versión alternativa con el camino más corto y el resultado fue idéntico, lo que indica que no hay caminos alternativos para esta arma en particular.

4.7. Consulta 7: proceso de fabricación más complejo

```

MATCH path = (base:Weapon)-[:TRANSFORMS*]->(target:Weapon)
WHERE EXISTS { (:WeaponCraft)-[:PRODUCES]->(base) }
AND NOT EXISTS { (:WeaponCraft)-[:PRODUCES]->(target) }

WITH target,
     [n IN nodes(path) WHERE n:Weapon | n.name] AS armas,
     size([n IN nodes(path) WHERE n:Weapon]) - 1 AS numero_de_pasos

ORDER BY numero_de_pasos DESC
LIMIT 1

RETURN target.name AS arma,
       target.kind AS tipo,
       armas AS armas_intermedias,
       numero_de_pasos

```

Se buscan caminos de actualización desde armas crafteables hasta armas objetivo. El NOT EXISTS sobre el target asegura que nos centramos en armas que solo se obtienen por actualización. Para contar los pasos se usa `size([n IN nodes(path) WHERE n:Weapon]) - 1`, que es más preciso que `length(path)` porque el camino puede incluir nodos `WeaponUpgrade`.

Resultado: el arma con el proceso más complejo es **Bors leal**, de tipo **gunlance**, con **6 pasos de actualización**. La lista de armas intermedias muestra toda la cadena que hay que seguir desde el arma base hasta llegar a ella. Es notable que el tipo **gunlance** sea el que tiene tanto el mayor daño combinado (consulta 4) como el proceso de fabricación más largo, lo que lo convierte en el arma más exigente del juego en todos los sentidos.

4.8. Consulta 8: arma que requiere matar más monstruos distintos

```

MATCH path = (wc:WeaponCraft)-[:PRODUCES]->(base:Weapon)
              -[:TRANSFORMS*0..]->(target:Weapon)

UNWIND [n IN nodes(path) WHERE n:WeaponCraft OR n:WeaponUpgrade] AS step

MATCH (step)-[:NEEDS]->(:Item)<-[:DROPS]-(m:Monster)

WITH target.name AS nombre,
     [n IN nodes(path) WHERE n:Weapon | n.name] AS armas,
     collect(DISTINCT m.name) AS monstruos

RETURN nombre,
       armas,
       monstruos,
       size(monstruos) AS numero_monstruos

ORDER BY numero_monstruos DESC
LIMIT 1

```

Se usan `TRANSFORMS*0..` para contemplar también armas que no necesitan ningún upgrade y se incluye el `WeaponCraft` en el path para no perder los materiales de la creación inicial. Se usa `collect(DISTINCT m.name)` porque el enunciado pide monstruos distintos: si el mismo monstruo dropea varios materiales, solo debe contarse una vez.

Resultado: el arma es **Arco recio de cazador**, con una cadena que empieza en **Arco de cazador I**. El proceso completo requiere matar **13 monstruos diferentes**. Es el proceso más exigente en variedad de monstruos del juego, lo que contrasta con el *Bors leal* de la consulta 7: ese tiene mas pasos de actualización pero necesita menos tipos de monstruo, mientras que el arco

requiere cazar una gran variedad de criaturas aunque su cadena sea más corta.

4.9. Consulta 9: debilidad máxima por localización

```
MATCH (l:Location)<-[:LIVES_IN]-(m:Monster)-[w:WEAK_TO]->(e:Element {
  kind: 'element'})

WITH l,
  e.name AS elemento,
  sum(w.level) AS debilidadTotal

WITH l,
  collect({elemento: elemento, debilidad: debilidadTotal}) AS
  debilidades,
  max(debilidadTotal) AS maxDebilidad

RETURN l.name AS ubicacion,
  maxDebilidad AS debilidadMaxima,
  [d IN debilidades WHERE d.debilidad = maxDebilidad | d.elemento]
  AS elementos

ORDER BY ubicacion
```

Para cada localización se suman los niveles de debilidad de todos sus monstruos agrupados por elemento. Después se calcula la debilidad máxima y se devuelven todos los elementos que alcanzan ese valor, gestionando posibles empates con comprensión de listas.

Resultado: Ruinas de Wyveria tiene la mayor debilidad acumulada del juego: **8** frente a fire. Acantilados del Tempano tiene **6** también frente a fire, lo que convierte el fuego en el elemento más efectivo del juego en general. Hay localizaciones con empates entre elementos: Bosque Escarlata es igualmente vulnerable a [fire, dragon], Cuenca Oleosa a [water, dragon] y Llanos Barlovento a [fire, thunder, dragon]. Estos empates confirman que la consulta gestiona correctamente ese caso con la comprensión de listas.

4.10. Consulta 10: recomendación de arma por localización

```
MATCH (l:Location)<-[:LIVES_IN]-(m:Monster)-[weak:WEAK_TO]->(e:Element {
  kind: 'element'})

WITH l, e,
  sum(weak.level) AS debilidadTotal

WITH l,
  collect({elemento: e, nombre: e.name, debilidad: debilidadTotal})
  AS debilidades,
  max(debilidadTotal) AS maxDebilidad

WITH l,
  [d IN debilidades WHERE d.debilidad = maxDebilidad | d.elemento] AS
  elementosVulnerables,
  [d IN debilidades WHERE d.debilidad = maxDebilidad | d.nombre] AS
  nombresElementos

MATCH (arma:Weapon {kind: $tipoArma})
OPTIONAL MATCH (arma)-[deal:DEALS]->(e:Element {kind: 'element'})

WITH l, nombresElementos, arma,
```

```

sum(
  CASE
    WHEN e IN elementosVulnerables THEN coalesce(deal.damage, 0)
    ELSE 0
  END
) AS danoContraElementos

WITH l, nombresElementos,
  collect({arma: arma.name, dano: danoContraElementos}) AS armas,
  max(danoContraElementos) AS maxDano

RETURN l.name AS ubicacion,
  nombresElementos AS elementosVulnerables,
  [a IN armas WHERE a.dano = maxDano | a.arma] AS armasRecomendadas

ORDER BY ubicacion

```

Esta consulta recomienda armas de un tipo dado como parametro. Reutiliza la lógica de la consulta 9 para identificar los elementos más efectivos en cada zona. La parte clave es el **CASE** que solo suma el daño elemental de un arma si el elemento coincide con la debilidad de la zona; si no coincide, suma 0. Se probó con el tipo **great-sword**.

Resultado: para **great-sword**, la consulta recomienda **Decapitagallos** en la mayoría de localizaciones porque hace buen daño de **fire**, que aparece como elemento vulnerable en varias zonas. Sin embargo, en **Cuenca Oleosa**, donde los elementos mas vulnerables son **[water, dragon]**, la recomendación cambia a **Lamorak reforzado**, que tiene daño contra esos elementos. Esto demuestra que la consulta adapta la recomendación a las características de cada localización en lugar de devolver siempre el arma con mas daño bruto.

5. Análisis de grafos con Graph Data Science

La segunda parte de la práctica utiliza el cliente Python de *Graph Data Science* (GDS 2.12.0). El objetivo es agrupar armas según sus materiales de fabricación o actualización y obtener un representante por comunidad.

5.1. Paso 1: creación de relaciones SIMILAR con índice de Jaccard

Antes de crear las relaciones se limpian posibles ejecuciones anteriores con `FOREACH (r IN relaciones | DELETE r)`, usando `collect(DISTINCT s)` para evitar errores con relaciones bi-direccionales.

Se crea una relación **SIMILAR** entre dos armas si comparten al menos un material, tanto de crafeo (**WeaponCraft**) como de mejora (**WeaponUpgrade**). El peso de la relación se calcula con el **índice de Jaccard**:

$$Jaccard(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

donde A y B son los conjuntos de materiales de cada arma. Para evitar duplicados se usa `WHERE id(w1) < id(w2)`, que garantiza que cada par se procesa una sola vez.

```
# Fragmento clave del pipeline Cypher para crear SIMILAR
WITH w1, w2, items1, items2,
     [i IN items1 WHERE i IN items2] AS interseccion,
     items1 + [i IN items2 WHERE NOT i IN items1] AS unionItems
WHERE size(interseccion) > 0
WITH w1, w2,
     toFloat(size(interseccion)) / size(unionItems) AS jaccard
MERGE (w1)-[s:SIMILAR]->(w2)
SET s.weight = jaccard
```

Resultados de la creación:

relacionesCreadas	pesoMínimo	pesoMáximo	pesoMedio
28 140	0.125	1.0	0.322835

Se crearon 28 140 relaciones **SIMILAR**. El peso mínimo de 0.125 significa que las armas menos similares solo comparten un octavo de sus materiales. El peso máximo de 1.0 indica que existen pares de armas con exactamente los mismos materiales, lo que puede deberse a armás de distintos tipos que usan las mismas materias primas. El peso medio de 0.32 refleja que la similitud típica entre armás conectadas es moderada. Se verifico además que no habia pesos invalidos ni relaciones de un arma consigo misma, confirmando la correccion del cálculo.

5.2. Paso 2: proyección del grafo y detección de comunidades con Leiden

Para aplicar algoritmos GDS se proyecta el grafo en memoria con nodos **Weapon** y relaciones **SIMILAR** como no dirigidas, ya que la similitud no tiene direccion real:

```
G, resultado_proyeccion = gds.graph.project(
    nombre_grafo,
    "Weapon",
    {
        "SIMILAR": {
            "orientation": "UNDIRECTED",
            "properties": "weight"
        }
    }
)
```

)

Resultado de la proyeccion:

Nodos	Relaciones
1 024	56 280

Aunque en Neo4j se crearon 28 140 relaciones dirigidas, GDS las duplica internamente al proyectarlas como no dirigidas, dando 56 280. Esto es correcto y esperado.

Después se aplica el **algoritmo de Leiden**:

```
resultado_leiden = gds.leiden.write(  
    G,  
    writeProperty="comunidad",  
    relationshipWeightProperty="weight",  
    randomSeed=19  
)
```

Leiden agrupa las armas maximizando la modularidad del grafo. La propiedad **weight** hace que las armas con mayor similitud de Jaccard tengan más influencia al formar sus comunidades. El resultado se escribe en cada nodo **Weapon** como la propiedad **comunidad**.

Resultados de Leiden:

Propiedad	Valor
Nodos escritos	1 024
Comunidades	58
Convergencia	True
Niveles ejecutados	3
Modularidad	0.7906

La modularidad de **0.79** es alta: indica que las armas dentro de cada comunidad comparten muchos más materiales entre si que con armas de otras comunidades. El algoritmo convergio en solo 3 niveles, lo que señala que la estructura de comunidades del grafo es clara y bien definida. Se detectaron 58 comunidades para 1 024 armas, lo que da una media de unos 17–18 armas por comunidad, aunque la distribucion real es muy heterogenea: hay comunidades grandes y comunidades de una sola arma.

5.3. Paso 3: PageRank por comunidad

Se aplica PageRank dentro de cada comunidad para identificar las armas más centrales (representantes). Para ello se crea un subgrafo por comunidad con **gds.graph.filter**:

```
query_filtrar_subgrafo = """  
CALL gds.graph.filter(  
    $subgraphName,  
    $graphName,  
    $nodeFilter,  
    '*'  
)  
YIELD graphName, nodeCount, relationshipCount  
RETURN graphName, nodeCount, relationshipCount  
"""  
  
# nodeFilter del estilo: "n.comunidad = 35"
```

Después se ejecuta PageRank sobre cada subgrafo usando el peso Jaccard como propiedad de relacion:

```
df_pagerank = gds.pageRank.stream(  
    Gsub,  
    relationshipWeightProperty="weight"  
)
```

Las armas con mayor PageRank dentro de su comunidad se consideran representantes. Si varias armas empatan en el máximo, se devuelven todas.

Resultados: las comunidades tienen tamaños muy dispares. La comunidad **35** es la mas grande, con **128 armas y 16 032 relaciones internas**, formando un cluster muy denso de armas con muchos materiales en comun. La comunidad **50** le sigue con **100 armas y 9 900 relaciones**. En el otro extremo hay comunidades de **1 nodo y 0 relaciones**: estas son armas aisladas que no comparten suficientes materiales con ninguna otra arma para ser agrupadas. En esos casos, el PageRank toma el valor base **0.15** y esa única arma es automaticamente la representante de su comunidad. En algunas comunidades medianas aparecen varios representantes cuando varias armas tienen la misma centralidad, lo que ocurre cuando comparten patrones de conexión muy similares dentro del subgrafo.

6. Recomendaciones de diseño

La última parte analiza si tiene sentido convertir el atributo `kind` de `Weapon` en un nodo independiente `WeaponKind`.

El diseño actual representa el tipo como propiedad:

```
(:Weapon {kind: "sword"})
```

El diseño alternativo seria:

```
(:Weapon) -[:HAS_KIND] -> (:WeaponKind {name: "sword"})
```

La decision entre nodo y propiedad depende de cuatro criterios: cardinalidad, patrones de consulta, identidad y conectividad, y evolución del modelo.

6.1. Cardinalidad

La base de datos contiene **14 tipos de arma**, cada uno compartido por entre 64 y 79 armas. Mantener `kind` como propiedad string duplica el mismo valor en aproximadamente 70 nodos `Weapon` por tipo. Como nodo `WeaponKind`, ese valor existiría **una sola vez** y todas las armas lo referenciarían via `HAS_KIND`.

Además, si un mismo `Weapon` pudiera pertenecer a **más de un tipo** (por ejemplo, un arma hibrida espada-lanza), modelarlo como propiedad obligaría a almacenar listas dentro del nodo. Con `WeaponKind` como nodo, esta relación *many-to-many* se representa de forma natural con varias relaciones `HAS_KIND`, y habilita consultas del tipo *el arma que mas daño hace dentro de un tipo y que no pertenece a otro*.

6.2. Patrones de consulta

La jerarquía de acceso en Neo4j es NODO < RELACION < PROPIEDAD. El tipo de arma actúa como **punto de entrada o agrupador** en varias consultas de la práctica:

- **Consulta 4:** agrupa todas las armas por tipo para calcular el máximo daño. Con `WeaponKind` como nodo, la búsqueda arrancaría desde el nodo en lugar de escanear todos los `Weapon` filtrando por propiedad.
- **Consulta 10:** recibe un tipo de arma como parametro y recomienda armas por localización. El tipo es lo primero que se busca, por lo que tenerlo indexado como nodo evita recorrer todas las armas.

Mas alla de las consultas del enunciado, otros patrones típicos también se beneficiarían:

- **Recuperar todas las armás de un tipo:** en lugar de escanear todos los `Weapon` filtrando por `kind`, bastaría con localizar el nodo `WeaponKind` y seguir sus relaciones `HAS_KIND`.
- **Item mas utilizado para fabricar armás de un tipo:** el `WeaponKind` actuaría como agrupador natural y la consulta partía directamente desde el.
- **Devolver el tipo como entidad independiente:** poder retornarlo con sus propias propiedades sin tener que arrastrar siempre el `Weapon` asociado.

6.3. Identidad y conectividad

Una propiedad no puede tener relaciones. Si se quisiera conectar **WeaponKind** con **Element** (para saber que tipo de arma es más eficaz contra ciertos elementos), con **Monster** (debilidades por tipo de arma) o con **Location** (recomendaciones directas por bioma), **solo es posible si WeaponKind es un nodo**. Esto enriquecería consultas como la 9 y la 10 sin necesidad de recorridos largos ni de cargar logica fuera del grafo. En resumen: el cambio convierte a **WeaponKind** en un **nodo-puente** capaz de enlazar otras entidades, algo estructuralmente imposible con una propiedad.

6.4. Evolución del modelo

Si el juego creciera y los tipos de armas pasaran a tener información propia (subtipos, relaciones con monstruos, elementos, habilidades o reglas específicas de combate), tener **WeaponKind** como nodo permitiría modelar todo eso directamente con relaciones, sin necesidad de reestructurar el grafo ni migrar datos masivamente.

6.5. Conclusión: cuando está justificado el cambio

El cambio a **WeaponKind** está justificado por tres razones principales:

1. La **alta cardinalidad** (14 tipos, ~70 armas cada uno) y la posibilidad de modelar relaciones many-to-many eliminan la redundancia de almacenar el mismo string en decenas de nodos.
2. Su papel como **punto de entrada frecuente** en consultas (4, 10 y otras agrupaciones por tipo) justifica tenerlo como entidad indexable en el grafo.
3. La capacidad de **habilitar relaciones futuras** con **Element**, **Monster** o **Location**, imposibles con una propiedad, lo convierte en una mejora estructural con beneficio tanto presente como potencial.

En el estado actual del modelo, como **kind** funciona como categoría plana sin atributos ni relaciones propias, mantenerlo como propiedad es técnicamente suficiente. Pero dado que ya actúa como agrupador en varias consultas y el modelo tiene claro potencial de crecimiento, el cambio a nodo es la decisión más robusta a largo plazo.

7. Conclusiones

La práctica ha permitido aplicar Neo4j a un dominio con relaciones complejas como es el sistema de crafeo y mejora de armas de *Jotun's Lair*. Las consultas Cypher permiten navegar de forma natural por cadenas de fabricación, materiales, monstruos, elementos y localizaciones, con una expresividad que habría requerido múltiples JOINS en SQL.

Algunos puntos destacables:

1. Las consultas con caminos variables (**TRANSFORMS***) son especialmente potentes para modelar cadenas de actualización de longitud desconocida, algo complicado en un modelo relacional.
2. La correcta gestión de empates mediante comprensión de listas (**[a IN armas WHERE a.daño = maxDaño | a.nombre]**) resulta más limpia en Cypher que la alternativa SQL con subconsultas o **RANK()**.

3. El *Graph Data Science* añade una capa de análisis estructural sobre el grafo que va más allá de las consultas tradicionales: la combinación de Jaccard, Leiden y PageRank permite descubrir grupos de armas similares y sus representantes de forma totalmente automática, sin necesitar lógica de negocio externa.
4. La reflexión de diseño muestra que en Neo4j la decisión de modificar un atributo debe tomarse en función de su conectividad futura y su papel en las consultas, no solo de si actualmente tiene atributos propios.